

HW 04 - Modeling the GSS

Moriah Ruggerio



Figure 1: Photo Mauro Mora on Unsplash

In this assignment we continue our exploration of the 2016 GSS dataset from the previous homework.

Getting started

Go to the course GitHub organization and locate your homework repo, clone it in RStudio and open the R Markdown document. Knit the document to make sure it compiles without errors.

Warm up

Before we introduce the data, let's warm up with some simple exercises. Update the YAML of your R Markdown file with your information, knit, commit, and push your changes. Make sure to commit with a meaningful commit message. Then, go to your repo on GitHub and confirm that your changes are visible in your Rmd **and** md files. If anything is missing, commit and push again.

Packages

We'll use the **tidyverse** package for much of the data wrangling and visualisation, the **tidymodels** package for modeling and inference, and the data lives in the **dsbox** package. These packages are already installed for you. You can load them by running the following in your Console:

```
library(tidyverse)
library(tidymodels)
library(dsbox)
```

Data

The data can be found in the **dsbox** package, and it's called **gss16**. Since the dataset is distributed with the package, we don't need to load it separately; it becomes available to us when we load the package. You can find out more about the dataset by inspecting its documentation, which you can access by running **?gss16** in the Console or using the Help menu in RStudio to search for **gss16**. You can also find this information [here](#).

Exercises

Scientific research

In this section we're going to build a model to predict whether someone agrees or doesn't agree with the following statement:

Even if it brings no immediate benefits, scientific research that advances the frontiers of knowledge is necessary and should be supported by the federal government.

The responses to the question on the GSS about this statement are in the **advfront** variable.

It's important that you don't recode the NAs, just the remaining levels.

1. Re-level the **advfront** variable such that it has two levels: **Strongly agree** and "Agree" combined into a new level called **agree** and the remaining levels (except NAs) combined into "Not agree". Then, re-order the levels in the following order: "Agree" and "Not agree". Finally, **count()** how many times each new level appears in the **advfront** variable.

```
#gss16 %>%
#count(advfront)           # pulls up exact spelling

gss16 <- gss16 %>%
  mutate(advfront = factor(advfront,
    c("Agree", "Strongly agree", "Disagree", "Dont know", "Strongly disagree", ""), # previous levels
    c("Agree", "Agree", "Not agree", "Not agree", "Not agree", "") # new levels
  ),
  advfront = factor(relevel(advfront, ref = "Agree")))

gss16 %>%
  count(advfront)
```

```
## # A tibble: 3 x 2
##   advfront      n
##   <fct>      <int>
## 1 Agree      1159
## 2 Not agree   226
## 3 <NA>       1482
```

You can do this in various ways. One option is to use the `str_detect()` function to detect the existence of certain words in the `polviews` variable.

- Combine the levels of the `polviews` variable such that levels that have the word “liberal” in them are lumped into a level called “Liberal” and those that have the word conservative in them are lumped into a level called “Conservative”. Then, re-order the levels in the following order: “Conservative”, “Moderate”, and “Liberal”. Finally, `count()` how many times each new level appears in the `polviews` variable.

```
gss16 <- gss16 %>%
  mutate(polviews = tolower(polviews), #converts all text to lowercase for searching
         polviews =
           if_else(str_detect(polviews, "liberal"), "Liberal", # if polviews contains "liberal" for tha
                 if_else(str_detect(polviews, "conservative"), "Conservative", # if polviews contains
                           if_else(polviews == "moderate", "Moderate", ""))), # converts "moderate" to
         polviews = factor(polviews, levels = c("Conservative", "Moderate", "Liberal"))) # reorders the

gss16 %>%
  count(polviews)
```

```
## # A tibble: 4 x 2
##   polviews      n
##   <fct>      <int>
## 1 Conservative  928
## 2 Moderate     1032
## 3 Liberal       796
## 4 <NA>         111
```

- Create a new data frame called `gss16_advfront` that includes the variables `advfront`, `educ`, `polviews`, and `wrkstat`. Then, use the `drop_na()` function to remove rows that contain NAs from this new data frame. Sample code is provided below.

```
gss16_advfront <- gss16 %>%
  select(advfront, educ, polviews, wrkstat) %>% #selects only relevant columns
  drop_na() # removes na
```

- Split the data into training (75%) and testing (25%) data sets. Make sure to set a seed before you do the `initial_split()`. Call the training data `gss16_train` and the testing data `gss16_test`. Sample code is provided below. Use these specific names to make it easier to follow the rest of the instructions.

```
set.seed(1122)

gss16_split <- initial_split(gss16_advfront, prop = 0.75)
gss16_train <- training(gss16_split)
gss16_test  <- testing(gss16_split)
```

- Create a recipe with the following steps for predicting `advfront` from `polviews`, `wrkstat`, and `educ`. Name this recipe `gss16_rec_1`. (We’ll create one more recipe later, that’s why we’re naming this recipe `_1`.) Sample code is provided below.

- `step_other()` to pool values that occur less than 10% of the time (`threshold = 0.10`) in the `wrkstat` variable into “Other”.

- `step_dummy()` to create dummy variables for `all_nominal()` variables that are predictors, i.e. `all_predictors()`

```
gss16_rec_1 <- recipe(advfront ~., data = gss16_train) %>% # creates a recipe for the model to predict
  step_other(wrkstat, threshold = 0.10, other = "Other") %>% # combines values that occur less than 10%
  step_dummy(all_nominal(), -all_outcomes())
```

- Specify a logistic regression model using "glm" as the engine. Name this specification `gss16_spec`. Sample code is provided below.

```
gss16_spec <- logistic_reg() %>%
  set_engine("glm") # sets engine to generalized linear model
```

- Build a workflow that uses the recipe you defined (`gss16_rec`) and the model you specified (`gss16_spec`). Name this workflow `gss16_wflow_1`. Sample code is provided below.

```
gss16_wflow_1 <- workflow() %>%
  add_model(gss16_spec) %>%
  add_recipe(gss16_rec_1)
```

- Perform 5-fold cross validation. specifically,

- split the training data into 5 folds (don't forget to set a seed first!),
- apply the workflow you defined earlier to the folds with `fit_resamples()`, and
- `collect_metrics()` and comment on the consistency of metrics across folds (you can get the area under the ROC curve and the accuracy for each fold by setting `summarize = FALSE` in `collect_metrics()`)
- report the average area under the ROC curve and the accuracy for all cross validation folds `collect_metrics()`

```
set.seed(365)
gss16_folds <- vfold_cv(gss16_train, v = 5) # sets the data and number of folds

gss16_fit_rs_1 <- gss16_wflow_1 %>%
  fit_resamples(gss16_folds) # performs the cross validation

collect_metrics(gss16_fit_rs_1, summarize = FALSE) %>%
  filter(.metric == "accuracy" | .metric == "roc_auc") # restricted to accuracy and area under the ROC
```

```
## # A tibble: 10 x 5
##   id      .metric .estimator .estimate .config
##   <chr> <chr>    <chr>      <dbl> <chr>
## 1 Fold1 accuracy binary      0.84 Preprocessor1_Model1
## 2 Fold1 roc_auc  binary      0.663 Preprocessor1_Model1
## 3 Fold2 accuracy binary      0.854 Preprocessor1_Model1
## 4 Fold2 roc_auc  binary      0.557 Preprocessor1_Model1
## 5 Fold3 accuracy binary      0.794 Preprocessor1_Model1
## 6 Fold3 roc_auc  binary      0.584 Preprocessor1_Model1
## 7 Fold4 accuracy binary      0.864 Preprocessor1_Model1
## 8 Fold4 roc_auc  binary      0.630 Preprocessor1_Model1
## 9 Fold5 accuracy binary      0.854 Preprocessor1_Model1
## 10 Fold5 roc_auc  binary      0.653 Preprocessor1_Model1
```

```
collect_metrics(gss16_fit_rs_1)
```

```
## # A tibble: 3 x 6
##   .metric      .estimator mean      n std_err .config
```

```
##   <chr>      <chr>      <dbl> <int>   <dbl> <chr>
## 1 accuracy   binary      0.841    5  0.0125 Preprocessor1_Model1
## 2 brier_class binary      0.132    5  0.0107 Preprocessor1_Model1
## 3 roc_auc    binary      0.618    5  0.0204 Preprocessor1_Model1
```

The folds were fairly accurate and consistent in their accuracy values ranging from 0.794 to 0.864. The area under ROC curve values for the fold were also roughly consistent ranging from 0.557 to 0.663. However, these values are quite low considering 0.5 is the probability of random guessing.

The average area under the ROC curve and accuracy was 0.618 and 0.841 respectively. While the model was fairly accurate, the area under the ROC curve was quite low.

9. Now, try a different, simpler model: predict `advfront` from only `polviews` and `educ`. Specifically,

- update the recipe to reflect this simpler model specification (and name it `gss16_rec_2`),
- redefine the workflow with the new recipe (and name this new workflow `gss16_wflow_2`),
- perform cross validation, and
- report the average area under the ROC curve and the accuracy for all cross validation folds `collect_metrics()`.

```
gss16_rec_2 <- recipe(advfront ~., data = gss16_train) %>% # creates a recipe for the model to predict
  step_rm(wrkstat) %>% # removes wrkstat
  step_dummy(all_nominal(), -all_outcomes())
```

```
gss16_wflow_2 <- workflow() %>%
  add_model(gss16_spec) %>%
  add_recipe(gss16_rec_2)
```

```
set.seed(365)
gss16_folds <- vfold_cv(gss16_train, v = 5) # sets the data and number of folds

gss16_fit_rs_2 <- gss16_wflow_2 %>%
  fit_resamples(gss16_folds) # performs the cross validation

collect_metrics(gss16_fit_rs_2)
```

```
## # A tibble: 3 x 6
##   .metric      .estimator mean      n std_err .config
##   <chr>      <chr>      <dbl> <int>   <dbl> <chr>
## 1 accuracy   binary      0.840    5  0.0134 Preprocessor1_Model1
## 2 brier_class binary      0.131    5  0.0103 Preprocessor1_Model1
## 3 roc_auc    binary      0.633    5  0.0191 Preprocessor1_Model1
```

10. Comment on which model performs better (one including `wrkstat`, model 1, or the one excluding `wrkstat`, model 2) on the training data based on area under the ROC curve.

Based on the area under the ROC curve, model 2, the one excluding `wrkstat`, (`roc_auc = 0.633`) performed slightly better as the value was closer to 1 than model 1 (`roc_auc = 0.618`).

11. Fit both models to the testing data, plot the ROC curves for the predictions for both models, and calculate the areas under the ROC curve. Does your answer to the previous exercise hold for the testing data as well? Explain your reasoning. Note: If you haven't yet done so, you'll need to first train your workflows on the training data with the following, and then use these fit objects to calculate predictions for the test data.

```
gss16_fit_1 <- gss16_wflow_1 %>%
  fit(gss16_train)
```

```

gss16_fit_2 <- gss16_wflow_2 %>%
  fit(gss16_train)

gss16_test_pred_1 <- predict(gss16_fit_1, new_data = gss16_test, type = "prob") %>% # predicts the prob
  bind_cols(gss16_test %>% select(advfront)) # outputs the predicted probabilities of Agree or Not agree

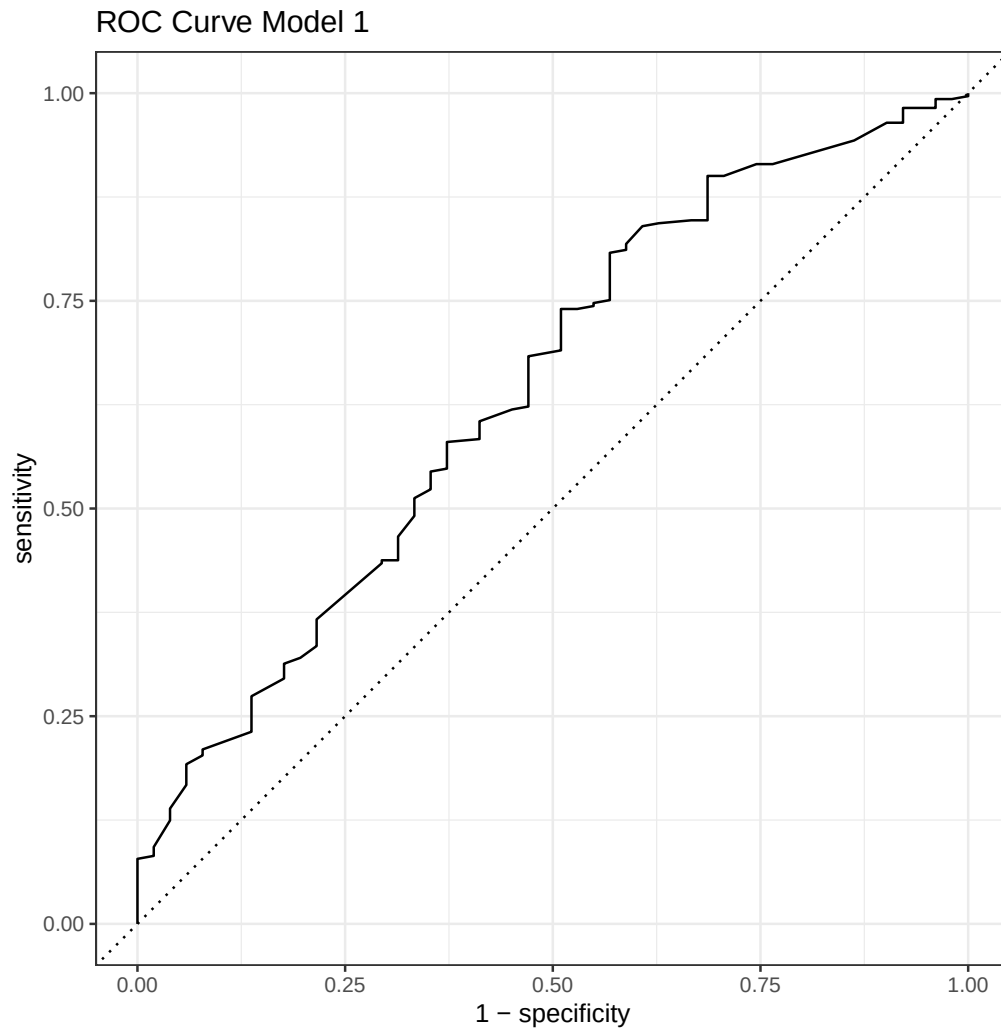
#gss16_test_pred_1

gss16_test_pred_1 %>%          # prints the area under the ROC curve value
  roc_auc(
    truth = advfront,
    .pred_Agree)              # predicted quantities

## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>       <dbl>
## 1 roc_auc binary      0.642

gss16_test_pred_1 %>%          # creates the curve
  roc_curve(
    truth = advfront,
    .pred_Agree) %>%          # predicted quantities
  autoplot() +
  labs(title = "ROC Curve Model 1")

```

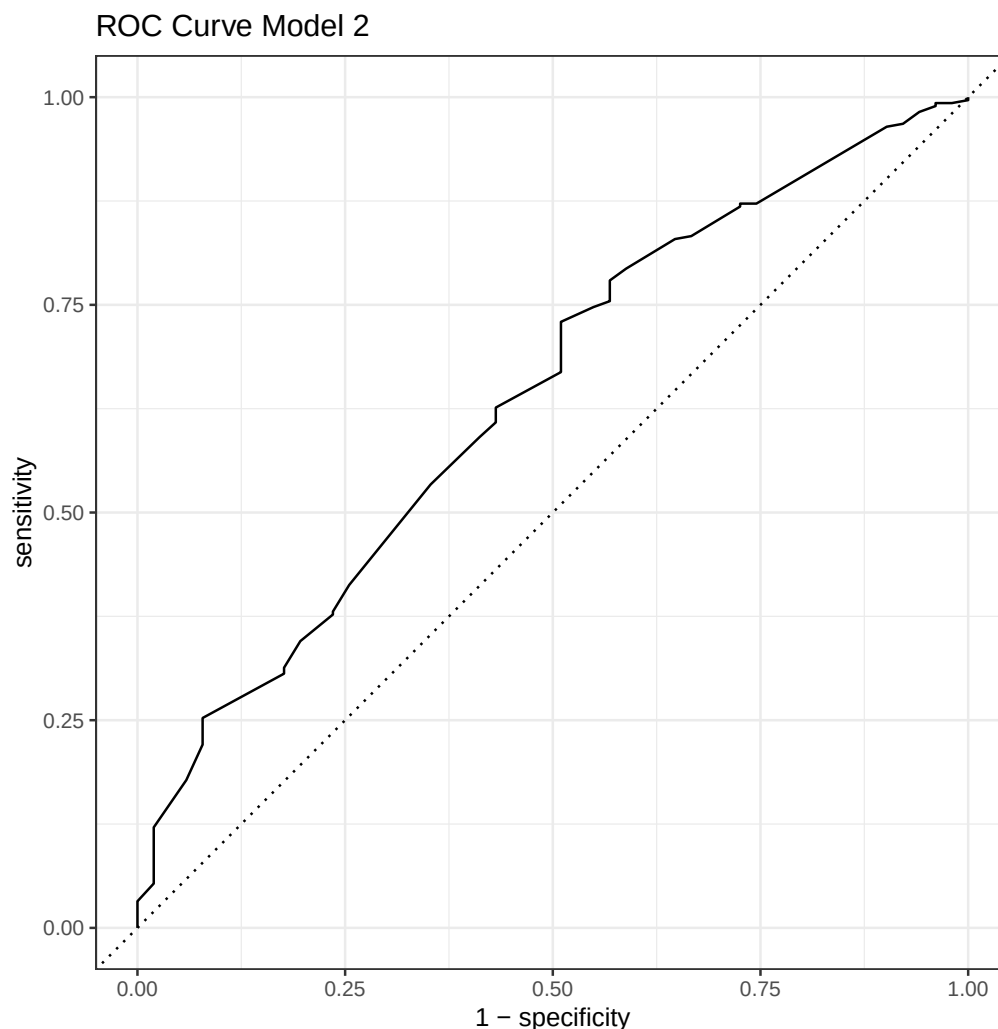


```
gss16_test_pred_2 <- predict(gss16_fit_2, new_data = gss16_test, type = "prob") %>% # predicts the prob
  bind_cols(gss16_test %>% select(advfront)) # outputs the predicted probabilities of Agree or Not agree
```

```
gss16_test_pred_2 %>% # prints the area under the ROC curve value
  roc_auc(
    truth = advfront,
    .pred_Agree) # predicted quantities
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>       <dbl>
## 1 roc_auc binary      0.639
```

```
gss16_test_pred_2 %>% # creates the curve
  roc_curve(
    truth = advfront,
    .pred_Agree) %>% # predicted quantities
  autoplot() +
  labs(title = "ROC Curve Model 2")
```



No, my answer to the previous exercise (model 2 performs better) does not hold on the testing data. On the testing data, model 1 ($\text{roc_auc} = 0.642$) performs better than model 2 ($\text{roc_auc} = 0.639$) as the area under the ROC curve is slightly closer to 1 for model 1.

Knit, commit, and push your changes to GitHub with an appropriate commit message. Make sure to commit and push all changed files so that your Git pane is cleared up afterwards.

Harassment at work

In 2016, the GSS added a new question on harassment at work. The question is phrased as the following.

Over the past five years, have you been harassed by your superiors or co-workers at your job, for example, have you experienced any bullying, physical or psychological abuse?

Answers to this question are stored in the `harass5` variable in our dataset.

12. Create a subset of the data that only contains **Yes** and **No** answers for the harassment question. How many responses chose each of these answers?

```
gss16_harass5 <- gss16 %>%
  filter(harass5 == "Yes" | harass5 == "No") %>% # only includes Yes and No answers
  select(harass5)                                # restricts to specified column
```



```
gss16_harass5 %>%
  count(harass5)
```

```
## # A tibble: 2 x 2
##   harass5     n
##   <chr>   <int>
## 1 No     1136
## 2 Yes    237
```

1136 respondents chose No, while 237 chose Yes.

13. Describe how bootstrapping can be used to estimate the proportion of Americans who have been harassed by their superiors or co-workers at their job.

Bootstrapping can be used to estimate the proportion of Americans who have been harassed by simulating the population parameter from the sample. Essentially, many random samples will be taken with replacement from the survey sample. This creates more samples like the one observed to calculate the proportion of harassment (bootstrap statistic). The proportion of responses for each bootstrapped sample is then plotted on a histogram to observe the distribution. From this distribution the bounds of the confidence interval can be calculated. This (the bounds of the confidence interval) gives a range of values for the proportion of workers who have experienced harassment to better estimate the population parameter.

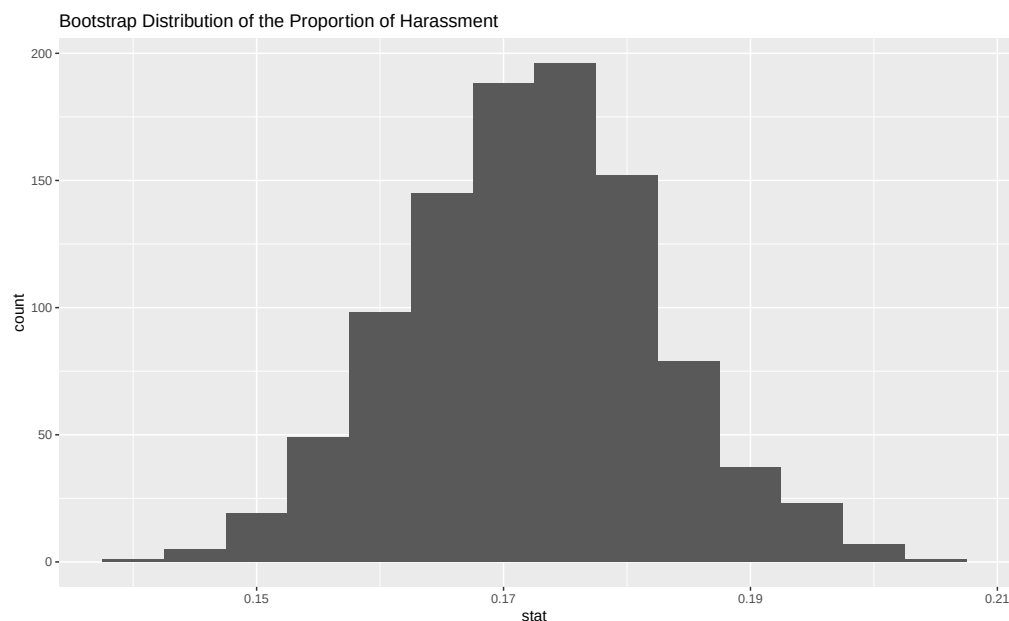
14. Calculate a 95% bootstrap confidence interval for the proportion of Americans who have been harassed by their superiors or co-workers at their job. Interpret this interval in context of the data.

```
gss16_harass5 <- gss16_harass5 %>%
  mutate(harass5 = factor(harass5, labels = c("Yes", "No")))
```

```
set.seed(165)
```

```
gss16_boot_df <- gss16_harass5 %>%
  # specify the variable of interest
  specify(response = harass5, success = "No") %>%
  # generate 1000 bootstrap samples
  # I tried to do the amount in the lecture but this is as much as my computer could keep up with knitt
  generate(reps = 1000, type = "bootstrap") %>%
  # calculate the mean of each bootstrap sample
  calculate(stat = "prop")
```

```
ggplot(data = gss16_boot_df, mapping = aes(x = stat)) +
  geom_histogram(binwidth = 0.005) +
  labs(title = "Bootstrap Distribution of the Proportion of Harassment")
```



```
gss16_boot_df %>%
  summarize(lower = quantile(stat, 0.025),
            upper = quantile(stat, 0.975))
```

```
## # A tibble: 1 x 2
##   lower upper
##   <dbl> <dbl>
## 1 0.153 0.194
```

We are 95% confident that the true proportion of Americans who have been harassed by their superiors or coworkers at their jobs is between 0.153 and 0.194.

15. Would you expect a 90% confidence interval to be wider or narrower than the interval you calculated above? Explain your reasoning.

I would expect a 90% confidence interval to be narrower than the interval I calculated above. Precision and accuracy have an inverse relationship in confidence intervals. When a confidence interval is more accurate, some precision is sacrificed. A less accurate confidence interval, therefore can be more precise.

Knit, commit, and push your changes to GitHub with an appropriate commit message. Make sure to commit and push all changed files so that your Git pane is cleared up afterwards and review the md document on GitHub to make sure you're happy with the final state of your work.